

Huffman 编码与解码模板

题目功能

给定若干字符及其权重，构造 Huffman 树，并支持：

- 输入普通字符串：输出对应 Huffman 编码；
- 输入由 0/1 组成的编码串：输出解码后的原字符串。

核心思路

Huffman 编码使用贪心策略：

- 每次从优先队列中取出权重最小的两个节点；
- 将它们合并成一个新节点；
- 新节点权重为两个节点权重之和；
- 重复直到只剩一个节点，即 Huffman 树根节点；
- 规定左边为 0，右边为 1，从根到叶子的路径就是字符编码。

Python 代码

```
import heapq

class HuffmanNode:
    def __init__(self, char, weight):
        self.left = None
        self.right = None
        self.char = char
        self.weight = weight

    def __lt__(self, other):
        """
        heapq 需要节点之间可以比较大小。

        优先按 weight 比较；
        如果 weight 相同，则按 char 字典序比较，保证结果稳定。
        """
        if self.weight == other.weight:
            return self.char < other.char
        return self.weight < other.weight

def main():
    n = int(input())
    heap = []
```

```
# 读取字符及权重，建立初始小根堆
for _ in range(n):
    char, weight = input().split()
    weight = int(weight)
    heap.append(HuffmanNode(char, weight))

heapq.heapify(heap)

# 构造 Huffman 树
while len(heap) > 1:
    left = heapq.heappop(heap)
    right = heapq.heappop(heap)

    # 合并节点：权重相加，char 用较小字符辅助比较
    merged = HuffmanNode(
        min(left.char, right.char),
        left.weight + right.weight
    )

    merged.left = left
    merged.right = right

    heapq.heappush(heap, merged)

root = heap[0]

# 存储每个字符对应的 Huffman 编码
codes = {}

def encode_tree(node, code):
    """
    DFS 遍历 Huffman 树，生成每个字符的编码。
    左子树走 0，右子树走 1。
    """
    if not node:
        return

    # 叶子节点代表一个字符
    if not node.left and not node.right:
        codes[node.char] = code
        return

    encode_tree(node.left, code + '0')
    encode_tree(node.right, code + '1')

encode_tree(root, '')

def huffman_encode(s):
    """
    将普通字符串编码成 Huffman 编码串。
    """
    result = ''
    for ch in s:
```

```
        result += codes[ch]
    return result

def huffman_decode(s):
    """
    将 0/1 编码串解码成原字符串。
    """
    result = ''
    node = root

    for bit in s:
        if bit == '0':
            node = node.left
        else:
            node = node.right

    # 到达叶子节点，解出一个字符
    if not node.left and not node.right:
        result += node.char
        node = root

    return result

result = []

# 持续读取直到 EOF
while True:
    try:
        line = input()

        # 如果输入全是数字，认为是 Huffman 编码串，进行解码
        if line.isdigit():
            result.append(huffman_decode(line))
        else:
            result.append(huffman_encode(line))

    except EOFError:
        break

for item in result:
    print(item)

if __name__ == '__main__':
    main()
```

调度场算法 Shunting Yard 模板

1. Tokenize：字符串切分

```
def tokenize(s):
    """
    把表达式字符串切成 token。
    例如：
    "12 + 3 * (4 - 5)" -> [12, '+', 3, '*', '(', 4, '-', 5, ')']
    """
    tokens = []
    i = 0
    n = len(s)

    while i < n:
        ch = s[i]

        if ch == ' ':
            i += 1
            continue

        if ch.isdigit():
            num = 0
            while i < n and s[i].isdigit():
                num = num * 10 + int(s[i])
                i += 1
            tokens.append(num)
            continue

        if ch in '+-*/()':
            tokens.append(ch)
            i += 1
            continue

        raise ValueError(f"非法字符：{ch}")

    return tokens
```

2. 中缀表达式转后缀表达式

完整版：支持一元正负号。

```
def infix_to_postfix(s):
    """
    调度场算法：中缀表达式 -> 后缀表达式
    支持：多位整数、空格、括号、+ - * /、一元正负号
    """
    tokens = tokenize(s)
```

```
priority = {
    '+': 1,
    '-': 1,
    '*': 2,
    '/': 2,
}

output = []
ops = []

# 用来判断 + / - 是一元还是二元
# 表达式开头、左括号后、运算符后出现的 + / - 是一元符号
prev = 'start'

for token in tokens:
    if isinstance(token, int):
        output.append(token)
        prev = 'num'

    elif token == '(':
        ops.append(token)
        prev = 'lparen'

    elif token == ')':
        while ops and ops[-1] != '(':
            output.append(ops.pop())

        if not ops:
            raise ValueError("括号不匹配")

        ops.pop() # 弹出 '('
        prev = 'num'

    else:
        # token 是 + - * /

        # 一元正负号:
        # -3 等价于 0 - 3
        # +3 等价于 0 + 3
        if token in '+-' and prev in ('start', 'op', 'lparen'):
            output.append(0)

        # 当前运算符入栈前，把栈顶优先级 >= 当前运算符的先弹出
        # 因为 + - * / 都是左结合
        while ops and ops[-1] != '(' and priority[ops[-1]] >= priority[token]:
            output.append(ops.pop())

        ops.append(token)
        prev = 'op'

while ops:
    if ops[-1] == '(':
        raise ValueError("括号不匹配")
```

```
output.append(ops.pop())
```

```
return output
```

3. 后缀表达式求值

```
def div_towards_zero(a, b):
    """
    整数除法：向 0 取整。
    LeetCode 224 / 227 / 772 这类题常用这个规则。
    """
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return abs(a) // abs(b) * (1 if a * b >= 0 else -1)

def eval_postfix(postfix):
    """
    后缀表达式求值
    """
    stack = []

    for token in postfix:
        if isinstance(token, int):
            stack.append(token)
        else:
            b = stack.pop()
            a = stack.pop()

            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            elif token == '/':
                stack.append(div_towards_zero(a, b))

    return stack[-1]
```

4. 完整 calculate 模板

```
def calculate(s):
    postfix = infix_to_postfix(s)
    return eval_postfix(postfix)

if __name__ == "__main__":
    s = input().strip()
    print(calculate(s))
```

5. 极简版： 题目保证没有一元负号时使用

如果题目保证没有 -3、(-2) 这种一元负号，可以用这个短版。

```
def infix_to_postfix_simple(s):
    priority = {'+': 1, '-': 1, '*': 2, '/': 2}
    output = []
    ops = []

    i = 0
    n = len(s)

    while i < n:
        ch = s[i]

        if ch == ' ':
            i += 1
            continue

        if ch.isdigit():
            num = 0
            while i < n and s[i].isdigit():
                num = num * 10 + int(s[i])
                i += 1
            output.append(num)
            continue

        if ch == '(':
            ops.append(ch)

        elif ch == ')':
            while ops and ops[-1] != '(':
                output.append(ops.pop())
            ops.pop()

        else:
            # ch 是 + - * /
            while ops and ops[-1] != '(' and priority[ops[-1]] >= priority[ch]:
                output.append(ops.pop())

            ops.append(ch)

        i += 1

    while ops:
        output.append(ops.pop())

    return output
```

```
ops.append(ch)

i += 1

while ops:
    output.append(ops.pop())

return output
```

中缀表达式编译成表达式树

最稳路线：

```
中缀表达式 -> 后缀表达式 -> 表达式树
```

6. 表达式树节点

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

7. 后缀表达式转表达式树

```
def postfix_to_tree(postfix):
    """
    后缀表达式 -> 表达式树
    """
    stack = []

    for token in postfix:
        node = TreeNode(token)

        if isinstance(token, int):
            stack.append(node)
        else:
            # 注意：先弹出的是右儿子，再弹出的是左儿子
            right = stack.pop()
            left = stack.pop()

            node.left = left
            node.right = right

            stack.append(node)

    return stack[-1]
```

8. 中缀表达式直接编译成树

```
def infix_to_tree(s):
    """
    中缀表达式 -> 表达式树
    """

    postfix = infix_to_postfix(s)
    return postfix_to_tree(postfix)
```

9. 表达式树遍历

```
def preorder(root):
    """
    前序遍历：根 左 右
    得到前缀表达式
    """

    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)


def inorder(root):
    """
    中序遍历：左 根 右
    得到中缀表达式
    为了避免歧义，加括号
    """

    if not root:
        return []

    if root.left is None and root.right is None:
        return [root.val]

    return ['('] + inorder(root.left) + [root.val] + inorder(root.right) + [')']


def postorder(root):
    """
    后序遍历：左 右 根
    得到后缀表达式
    """

    if not root:
        return []
    return postorder(root.left) + postorder(root.right) + [root.val]
```

10. 表达式树求值

```
def eval_tree(root):
    """
    表达式树求值
    """

    if root.left is None and root.right is None:
        return root.val

    a = eval_tree(root.left)
    b = eval_tree(root.right)

    if root.val == '+':
        return a + b
    elif root.val == '-':
        return a - b
    elif root.val == '*':
        return a * b
    elif root.val == '/':
        return div_towards_zero(a, b)
```

11. 完整整合版代码

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None


def tokenize(s):
    tokens = []
    i = 0
    n = len(s)

    while i < n:
        ch = s[i]

        if ch == ' ':
            i += 1
            continue

        if ch.isdigit():
            num = 0
            while i < n and s[i].isdigit():
                num = num * 10 + int(s[i])
                i += 1
            tokens.append(num)
            continue

        if ch in '+-*/()':
            tokens.append(ch)
            i += 1
```

```

        continue

    raise ValueError(f"非法字符: {ch}")

return tokens

def infix_to_postfix(s):
    tokens = tokenize(s)

    priority = {
        '+': 1,
        '-': 1,
        '*': 2,
        '/': 2,
    }

    output = []
    ops = []
    prev = 'start'

    for token in tokens:
        if isinstance(token, int):
            output.append(token)
            prev = 'num'

        elif token == '(':
            ops.append(token)
            prev = 'lparen'

        elif token == ')':
            while ops and ops[-1] != '(':
                output.append(ops.pop())

            if not ops:
                raise ValueError("括号不匹配")

            ops.pop()
            prev = 'num'

        else:
            if token in '+-' and prev in ('start', 'op', 'lparen'):
                output.append(0)

            while ops and ops[-1] != '(' and priority[ops[-1]] >= priority[token]:
                output.append(ops.pop())

            ops.append(token)
            prev = 'op'

    while ops:
        if ops[-1] == '(':
            raise ValueError("括号不匹配")

```

```

        output.append(ops.pop())

    return output

def div_towards_zero(a, b):
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return abs(a) // abs(b) * (1 if a * b >= 0 else -1)

def eval_postfix(postfix):
    stack = []

    for token in postfix:
        if isinstance(token, int):
            stack.append(token)
        else:
            b = stack.pop()
            a = stack.pop()

            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            elif token == '/':
                stack.append(div_towards_zero(a, b))

    return stack[-1]

def postfix_to_tree(postfix):
    stack = []

    for token in postfix:
        node = TreeNode(token)

        if isinstance(token, int):
            stack.append(node)
        else:
            right = stack.pop()
            left = stack.pop()

            node.left = left
            node.right = right

            stack.append(node)

    return stack[-1]

```



```
def infix_to_tree(s):
    postfix = infix_to_postfix(s)
    return postfix_to_tree(postfix)

def preorder(root):
    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

def inorder(root):
    if not root:
        return []

    if root.left is None and root.right is None:
        return [root.val]

    return ['('] + inorder(root.left) + [root.val] + inorder(root.right) + [')']

def postorder(root):
    if not root:
        return []
    return postorder(root.left) + postorder(root.right) + [root.val]

def eval_tree(root):
    if root.left is None and root.right is None:
        return root.val

    a = eval_tree(root.left)
    b = eval_tree(root.right)

    if root.val == '+':
        return a + b
    elif root.val == '-':
        return a - b
    elif root.val == '*':
        return a * b
    elif root.val == '/':
        return div_towards_zero(a, b)

def tokens_to_string(tokens):
    return ' '.join(map(str, tokens))

if __name__ == "__main__":
    s = input().strip()

    postfix = infix_to_postfix(s)
    root = postfix_to_tree(postfix)
```

```
print("postfix:", tokens_to_string(postfix))
print("prefix :", tokens_to_string(preorder(root)))
print("infix  :", tokens_to_string(inorder(root)))
print("value  :", eval_tree(root))
```

12. 测试样例

输入：

```
3 + 4 * 2 / (1 - 5)
```

输出大致为：

```
postfix: 3 4 2 * 1 5 - / +
prefix : + 3 / * 4 2 - 1 5
infix   : ( 3 + ( ( 4 * 2 ) / ( 1 - 5 ) ) )
value   : 1
```

13. 易错点

一元负号

```
-3 + 5
2 * (-3 + 4)
```

不能直接把 `-` 当普通二元减号，否则栈里会少一个左操作数。

常用处理方式：遇到一元 `-` 时，在前面补 `0`。

```
-3      -> 0 - 3
(-3)    -> (0 - 3)
2*(-3)  -> 2*(0 - 3)
```

建树弹栈顺序

后缀表达式遇到运算符时：

```
right = stack.pop()
left  = stack.pop()
```

先弹出来的是右儿子，后弹出来的是左儿子。这个别写反。